

Optimierer im Deep Learning

Skript – PyTorch-Referenz

Themen: Einführung in Optimierer, Übersicht der wichtigsten PyTorch-Optimierer, Abstammung und Zusammenhänge, Glossar der Fachbegriffe

1 Was ist ein Optimierer?

Ein Optimierer steuert, wie ein neuronales Netz aus seinen Fehlern lernt. Nach jedem Trainingsschritt berechnet die Backpropagation, wie stark jeder einzelne Parameter zum Gesamtfehler (Loss) beigetragen hat – das sind die Gradienten. Der Optimierer nutzt diese Gradienten, um die Parameter so anzupassen, dass der Loss im nächsten Schritt möglichst kleiner wird.

Die verschiedenen Optimierer unterscheiden sich darin, wie sie diese Anpassung vornehmen: Manche verwenden eine feste Schrittweite (Lernrate), andere passen sie pro Parameter dynamisch an oder berücksichtigen vergangene Gradienten, um Schwingungen zu dämpfen und schneller zu konvergieren.

In der Praxis hat sich gezeigt, dass die Wahl des Optimierers erheblichen Einfluss auf die Trainingsgeschwindigkeit und die Qualität des Endergebnisses hat. Die folgende Übersicht stellt die wichtigsten Optimierer in PyTorch vor, geordnet nach ihrer Praxisrelevanz.

2 Übersicht der Optimierer

Die folgende Tabelle listet die wichtigsten Optimierer aus `torch.optim` auf. Die Spalte „Praxisrelevanz“ gibt an, wie häufig der jeweilige Optimierer in aktuellen Projekten und Forschungsarbeiten eingesetzt wird.

Bezeichnung	Funktionsweise	Syntax	Praxisrelevanz
SGD	Aktualisiert die Parameter anhand des negativen Gradienten der Verlustfunktion, skaliert durch die Lernrate. Mit Momentum werden vergangene Gradienten berücksichtigt, was die Konvergenz beschleunigt.	<code>optim.SGD(params, lr=0.01)</code> <code>optim.SGD(params, lr=0.01, momentum=0.9)</code>	Sehr hoch – Standard in Computer Vision, besonders mit Momentum.
Adam	Kombiniert die Vorteile von Adagrad und RMSprop. Speichert laufende Schätzungen des Mittelwerts (1. Moment) und der Varianz (2. Moment) der Gradienten.	<code>optim.Adam(params, lr=0.001)</code>	Sehr hoch – beliebtester Optimierer insgesamt, häufiger Default.
AdamW	Entkoppelt die Gewichtsdekadenz von der Gradientenberechnung, was bei Adam fälschlicherweise vermischt wird. Führt zu korrekt wirkender Regularisierung. (<i>siehe unten</i>)	<code>optim.AdamW(params, lr=0.0001)</code>	Sehr hoch – Standard bei Transformer-Modellen (BERT, GPT, ViT).
RMSprop	Teilt die Lernrate durch einen exponentiell gewichteten Durchschnitt der quadrierten Gradienten, wodurch sie pro Parameter individuell skaliert wird.	<code>optim.RMSprop(params, lr)</code>	Mittel – historisch wichtig, v.a. bei RNNs. Durch Adam abgelöst.
Adagrad	Passt die Lernrate pro Parameter anhand der Summe aller bisherigen quadrierten Gradienten an. Die Lernrate sinkt monoton und kann zu früh gegen null gehen.	<code>optim.Adagrad(params, lr=0.01)</code>	Gering – nützlich bei dünn besetzten Daten, sonst selten.
NAdam	Integriert die Nesterov-Beschleunigung in Adams Momentum-Berechnung. Der Gradient wird an einer vorausschauenden Position berechnet, was die Konvergenz beschleunigen kann.	<code>optim.NAdam(params, lr)</code>	Gering – selten signifikant besser als Adam.
RAdam	Erkennt automatisch, wann die Varianzschätzung noch unzuverlässig ist, und schaltet in dieser Phase auf SGD um. Macht manuelles Warmup überflüssig.	<code>optim.RAdam(params, lr)</code>	Gering – gute Idee, aber explizites Warmup mit AdamW hat sich durchgesetzt.
Adadelat	Erweitert Adagrad, indem statt der gesamten Gradientenhistorie nur ein exponentiell gewichtetes Fenster verwendet wird. Benötigt keine manuell gesetzte Lernrate.	<code>optim.Adadelat(params, lr)</code>	Sehr gering – historisch relevant, heute kaum im Einsatz.

Hinweis: Selten verwendete Optimierer wie *SparseAdam* (nur für Sparse-Embeddings), *LBFGS* (Second-Order-Verfahren, im Deep Learning kaum relevant) und *Adamax* (durch Adam/AdamW ersetzt) wurden aus dieser Übersicht entfernt, da sie für den typischen Einsatz nicht praxisrelevant sind.

Zu AdamW:

Das Problem bei Adam: Man möchte, dass große Gewichte bei jedem Schritt ein kleines bisschen geschrumpft werden – das ist Weight Decay, eine Art eingebaute Bremse gegen Überanpassung. Bei Adam passiert dieses Schrumpfen aber nicht direkt an den Gewichten, sondern es wird in die Gradientenberechnung eingemischt. Das heißt, Adam behandelt die Bremse so, als wäre sie Teil des Lernens – und passt sie dann auch noch mit seiner adaptiven Lernrate an. Dadurch werden manche Gewichte zu stark gebremst und andere zu wenig.

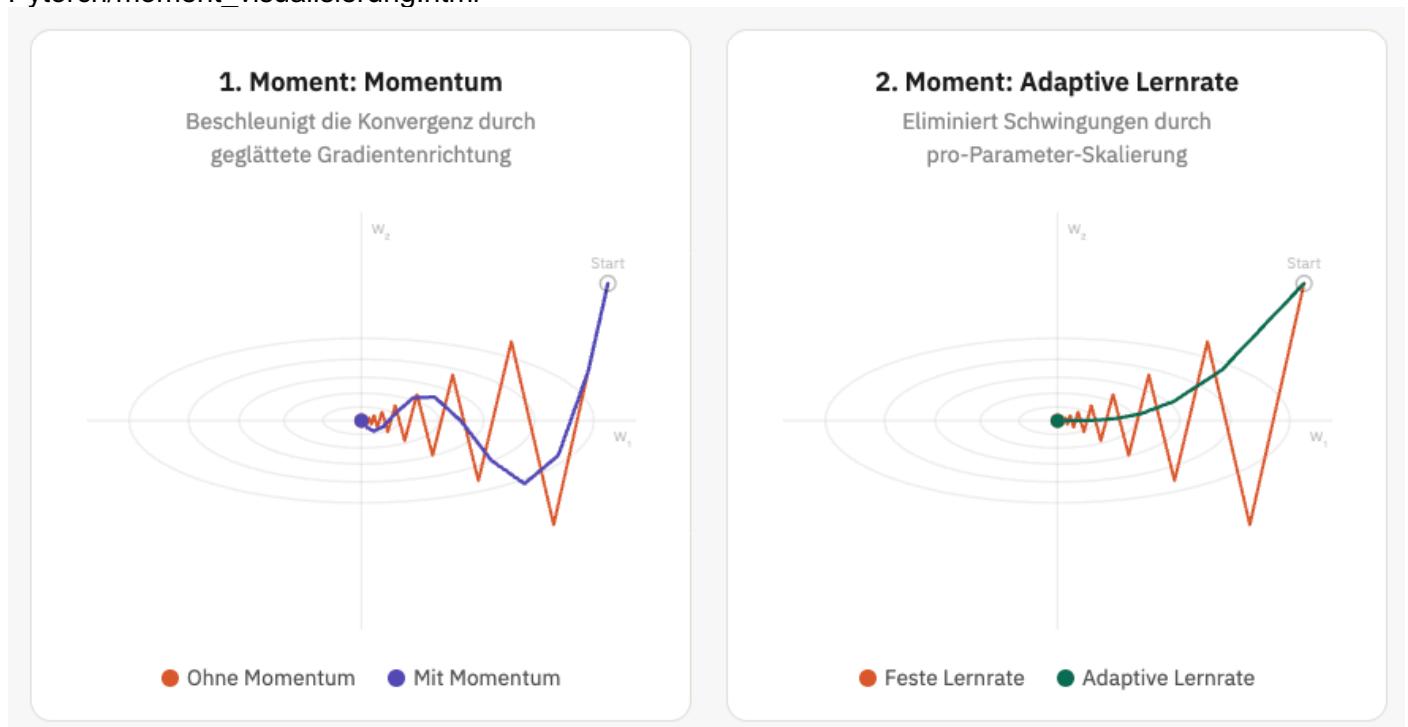
AdamW löst das, indem es die beiden Schritte trennt: Erst berechnet Adam ganz normal sein Update aus den Gradienten, und danach – als separaten Schritt – werden die Gewichte gleichmäßig geschrumpft. So wirkt die Bremse auf alle Gewichte gleich.

Visualisierung von Optimierern:

<https://emiliendupont.github.io/2018/01/24/optimization-visualization/>

<https://imgur.com/a/visualizing-optimization-algos-Hqolp>

Pytorch/moment_visualisierung.html



3 Abstammung der Optimierer

Die Entwicklung der Optimierer ist kein linearer Stammbaum, sondern ein gerichteter Graph. Der Grund: Adam entsteht als Kombination zweier unabhängiger Entwicklungslinien – des Momentum-Pfads und des adaptiven Lernraten-Pfads.

Der linke Zweig (Momentum-Pfad, violett) nutzt vergangene Gradienten, um Schwingungen zu dämpfen. Der rechte Zweig (adaptiver LR-Pfad, grün) passt die Schrittweite pro Parameter individuell an. Adam vereinigt beide Ideen, indem er sowohl den Mittelwert der Gradienten (1. Moment, vom Momentum-Pfad) als auch deren Varianz (2. Moment, vom RMSprop-Pfad) speichert.

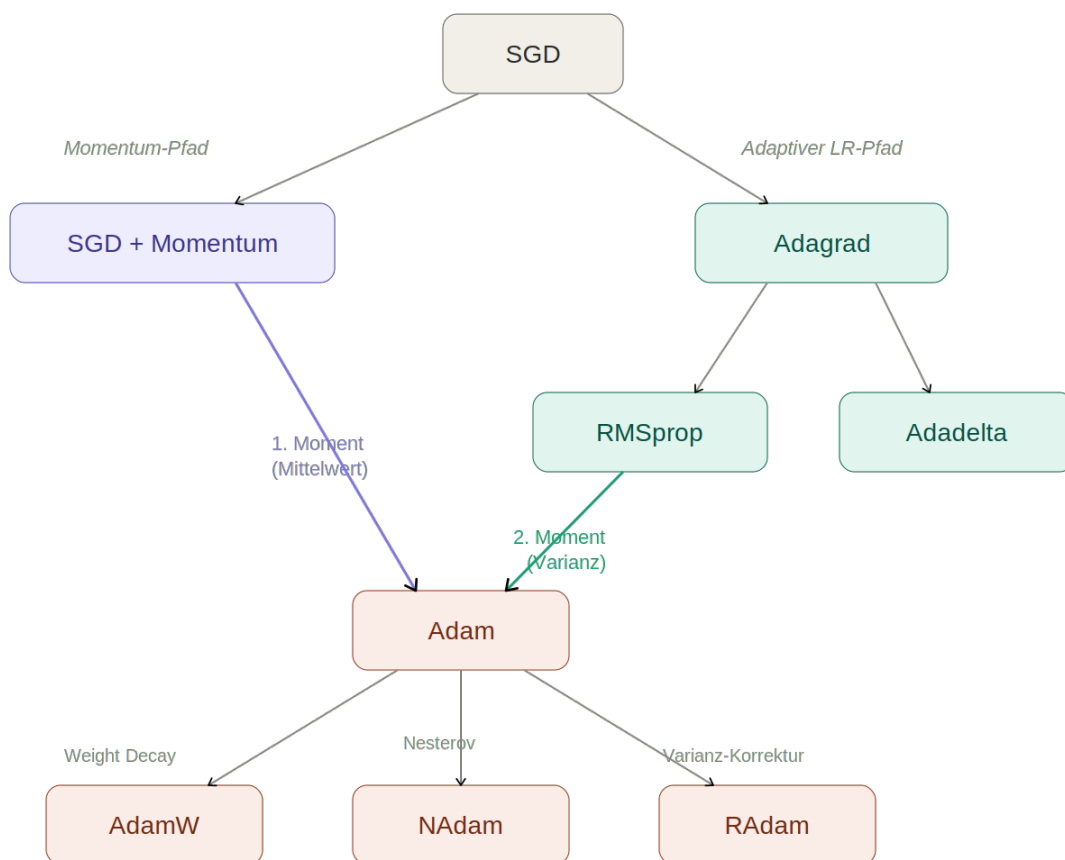


Abbildung 1: Abstammung der PyTorch-Optimierer. Adam entsteht als Zusammenführung des Momentum- und des adaptiven Lernraten-Pfads.

Adadelta ist ein Seitenzweig, der aus Adagrad hervorging, aber keinen Nachfolger in der Adam-Familie hat. Die drei Adam-Varianten ergänzen jeweils einen spezifischen Aspekt: AdamW korrigiert die Gewichtsdekadenz, NAdam fügt Nesterov-Vorausschau hinzu, und RAdam stabilisiert die frühe Trainingsphase.

4 Glossar der Fachbegriffe

Die folgende Tabelle erklärt die wichtigsten Fachbegriffe, die im Kontext der Optimierer auftreten. Die Begriffe sind thematisch gruppiert und folgen der Struktur des Abstammungsgraphen.

Begriff	Erklärung
Grundlagen	
Gradient	Ein Vektor, der für jeden Parameter angibt, in welche Richtung und wie stark der Loss steigt. Der Optimierer geht in die entgegengesetzte Richtung.
Gradientenabstieg	Das Grundprinzip aller Optimierer: Parameter werden schrittweise entgegen dem Gradienten verschoben, um den Loss zu verringern.
Lernrate (lr)	Die Schrittweite bei jedem Update. Zu groß → Training springt über das Minimum. Zu klein → extrem langsame Konvergenz.
Verlustfunktion (Loss)	Misst, wie weit die Vorhersage des Modells vom gewünschten Ergebnis entfernt ist. Das Training zielt darauf ab, diesen Wert zu minimieren.
Parameter	Die lernbaren Gewichte und Biases eines Netzes – die Zahlen, die der Optimierer bei jedem Schritt anpasst.
Backpropagation	Berechnet, wie stark jeder Parameter zum Gesamtfehler beiträgt. Liefert die Gradienten für den Optimierer.
Konvergenz	Der Zustand, in dem der Loss sich einem stabilen Minimum nähert und die Parameter sich kaum noch ändern.
Epoche	Ein vollständiger Durchlauf durch den gesamten Trainingsdatensatz. Pro Epoche sieht das Modell jeden Datenpunkt einmal.
Momentum & Beschleunigung	
Momentum	Speichert vergangene Gradienten und lässt sie in die aktuelle Richtung einfließen – wie ein Ball, der Schwung aufnimmt. Dämpft Schwingungen.
Nesterov-Beschleunigung	Variante von Momentum: Der Gradient wird an einer vorausgeschauten Position berechnet, was früheres Korrigieren ermöglicht.
Adaptive Methoden	
Adaptive Lernrate	Statt einer festen Lernrate für alle Parameter wird sie pro Parameter individuell angepasst – seltene Updates bekommen größere Schritte.
1. Moment (Mittelwert)	Exponentiell gewichteter Durchschnitt der Gradienten. Geglättete Richtungsschätzung, bei Adam als Momentum-Komponente genutzt.
2. Moment (Varianz)	Exponentiell gewichteter Durchschnitt der quadrierten Gradienten. Schätzt die Schwankungsstärke, skaliert die Lernrate pro Parameter.
Exp. gew. Durchschnitt	Jüngere Werte werden stärker gewichtet als ältere. Gesteuert durch β (z.B. 0.9): Je näher an 1, desto länger wirken alte Werte nach.
Regularisierung & Stabilisierung	
Weight Decay	Bei jedem Update werden alle Gewichte leicht Richtung Null geschrumpft. Verhindert extrem große Gewichte, verbessert Generalisierung.
Regularisierung	Oberbegriff für Techniken, die Überanpassung (Overfitting) verhindern. Weight Decay, Dropout und Data Augmentation sind Beispiele.

Begriff	Erklärung
Warmup	Phase zu Beginn des Trainings, in der die Lernrate langsam hochgefahren wird. Verhindert instabile Updates bei ungenauen Gradientenschätzungen.
Generalisierung	Die Fähigkeit eines Modells, auf neuen, ungesehenen Daten gut zu funktionieren – das eigentliche Ziel des Trainings.
Fortgeschrittene Konzepte	
Hesse-Matrix	Matrix der zweiten Ableitungen der Verlustfunktion. Beschreibt die Krümmung der Loss-Landschaft. Von L-BFGS approximiert genutzt.
Loss-Landschaft	Bildliche Vorstellung: Der Loss als Hügellandschaft über dem Raum aller Parameterwerte. Training = Suche nach dem tiefsten Tal.
Bias-Korrektur	Korrekturschritt in Adam: Da die Momentschätzungen mit Null starten, sind sie anfangs systematisch zu niedrig. Die Korrektur gleicht das aus.